then the calls to data gatherer fails for some reason, the net result will be that both web calls to our web server will fail.

That said, we can and need to try to do better and avoid waste of resources. To handle the situation described in the last sub-section, one thing we could do is certainly adding a “loading” state for cache values, used to mark entries in the cache that are currently being computed. When the first call to `/sentiment/Google` checks the cache and finds a miss, it will create an entry for Google and mark it as “in progress”.

When the second call checks the cache, it will be aware that the value is being computed. Then we can either do some time-based polling of the cache, checking every few hundreds of milliseconds until a value is stored for that entry, or implement a publisher-subscriber mechanism, with the web app subscribing to the cache to let it know its interested for the entry “Google”, and the cache notifying all subscribers for that entry once its value is finally computed.

Either way, we can’t completely solve the problems with the race conditions above, but risking a little extra delay and a few more 500s responses is in many cases worth the saving we earn by avoiding re-computing values that are in progress.

7.3 Memory Is Not Enough (Literally)

Let’s stop for a minute and recap what we have discussed so far: